

FINAL REPORT

ASSIGNMENT 1

NETWORKS

CSC3002F

Prepared by
Anele Ngxongo
Kamva Poswa
Thobelani Vilakazi

Chat77

Connect. Chat. Together.

ngxane002@myuct.ac.za
pswkam001@myuct.ac.za
vlktho006@myuct.ac.za

Final Report

Content:

Introduction	3
Architecture Overview	3
Protocol Specification and System Functions	6
Implementation Challenges	11
Performance and Reliability Discussion	12
Basic Security Considerations	13
Creativity ..	13
Conclusion	14

Introduction

The objective of this project was to design, prototype, and implement a networked chat application capable of supporting multiple communication features. These include one-to-one messaging, group conversations, and the exchange of arbitrary media such as images, audio, and video files. The system was also required to demonstrate the use of both Transmission Control Protocol (TCP) and User Datagram Protocol (UDP) transport protocols, as well as both client-server and peer-peer communication paradigms.

The resulting system named **Chat77**, implements a hybrid networking architecture in which a central server coordinates communication between multiple client applications. Reliable communication and message exchange are handled using TCP, while lightweight notifications are delivered using UDP. The application additionally supports authentication, private messaging, group chat management and file transfer.

This report describes the design, implementation, and evaluation of the Chat77 system. The report first presents the system architecture and protocol specification, followed by a discussion of implementation challenges. Finally, the performance, reliability, and security considerations of the system are analysed also creativity to capitalize usability.

Architecture Overview

The Chat77 application is designed using a hybrid client-server architecture with elements that support peer-to-peer components (such as private texting between two peers, private group chat with peers). The core logic, message routing, and user management are centralized on a server, while a separate out-of-band UDP channel is used for notifications.

Roles:

- **Central Server:** The server is the heart of the system. It maintains the state of the application, including the list of connected clients, user authentication credentials, group memberships, and private chat relationships. All text messages and file transfer data are routed through this server.
- **Clients:** Client applications connect to the server to authenticate and participate in chat features. They handle user input, display messages, and manage the local assembly of incoming file transfers.
- **Peer-to-Peer (P2P):** This is a communication model between peers connected directly to each other, rather than relying on a server. In our system, we implemented p2p mainly on logical functions, like private channel communication, request/accept of handshakes, file transfer, and beep notifications.
- **TCP (Reliable, Ordered Data Transfer):** The primary communication channel (**SOCK_STREAM**) between the client and server is TCP (Transmission Control Protocol). This choice is fundamental for a chat application where the order and delivery of messages must be guaranteed. All reliable data, including authentication credentials, chat logs, and file transfers are sent over this connection. TCP's congestion control and reliability mechanisms (as discussed in Chapter 3 of

Computer Networks A Top-Down Approach) ensure that messages arrive complete and in order, which is critical for the integrity of conversations and files.

- **UDP (Lightweight, Notification Beep):** A secondary, separate UDP (User Datagram Protocol) socket (**SOCK_DGRAM**) is used for the "beep" notification system. This is an example of choosing a transport protocol for its specific characteristics. The "beep" is a small and stateless notification. If a UDP packet is lost, the user simply misses a notification but will still receive the full message over TCP as it is reliable. This design offloads trivial notifications from the main TCP channel without the overhead of connection establishment.

Architecture System Diagram:

1. Structural Diagram

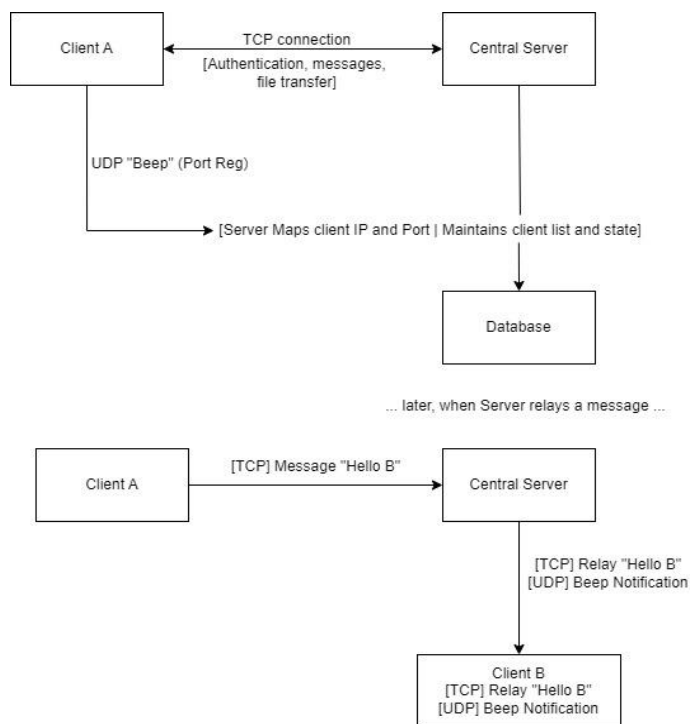


Figure 1: A structural diagram that communication between Client, Server and Database

2. Sequence Diagram

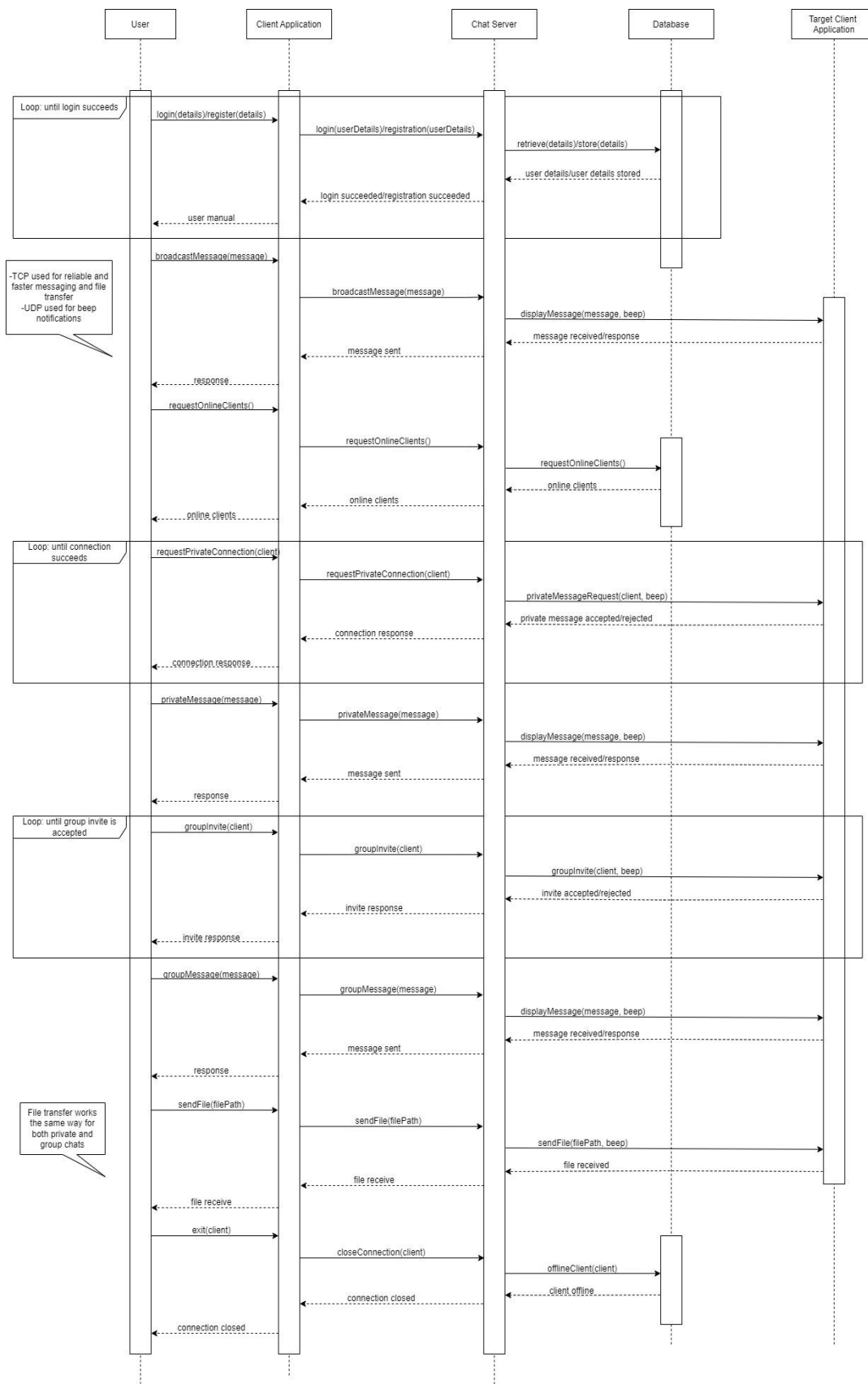


Figure 2: Sequence diagram that shows the flow of protocol as the interface is in usage.

Protocol Specification and System Functions

Chat77 uses a custom application-layer protocol to coordinate communication between clients and the server. This protocol defines the format and sequence of messages exchanged during system operation.

Message Types and Structure

The protocol consists of several categories of messages corresponding to different stages of communication.

Authentication Phase

During authentication, the server guides the client through a structured login or registration process.

Protocol messages include:

- AUTHORISE MODE? (REGISTER/LOGIN)

```
Enter server IP address: 192.168.171.52
Choose auth mode (REGISTER/LOGIN): login
Enter alias name: kamva
Enter password: kamva1
Authentication successful, Welcome to Chat77!
User Commands:
```

These messages establish a valid user session before any chat functionality is enabled.

User Commands Phase

After authentication, clients may issue various commands to interact with the system.

Broadcast Messaging

Users may send a broadcast message to all connected users using the command:

`bdct txt {message}`

```
bdct txt good morning guys, have you submitted?
```

```
[Beep] anele sent a message in BROADCAST
anele: good morning guys, have you submitted?
```

The server distributes the message to all active clients.

Private Chat Establishment

Private messaging requires the creation of a connection between two users through TCP

`connect to [client]` – `connect to anele`

`reject connection [client]` -

```
Private request from hope. Type: accept connection hope or reject connection hope
reject connection hope
INFO: Rejected private request from hope
```

`connection [client]` -

`accept`

```
Private request from hope. Type: accept connection hope or reject connection hope
accept connection hope
Private chat connected with hope. Use: private txt hope {your message}
```

Once both users have agreed to the connection, private messages can be exchanged.

Private Messaging

`private txt [client] {message}`

```
private txt hope hi friend
```

The server ensures that messages are only delivered between mutually connected users.

Group Communication

Group chat functionality allows multiple users to participate in shared conversations.

Key commands include:

- `create group [group_name]`

```
create group sportcentre
```

- `invite group [group_name] [client]`

```
invite group sportcentre hope
INFO: Group invite sent to hope for sportcentre
```

- `accept group [group_name]`

```
Group invite to 'sportcentre' from lwandile. Type: accept group sportcentre or reject group sportcentre
accept group sportcentre
You joined group: sportcentre
```

- `group txt [group_name] {message}`

```
group txt sportcentre hi guys, nippi?
```

The server maintains group membership information and distributes messages to all group members.

File Transfer

The system also supports the transfer of media files.

File transfers follow a chunk-based protocol consisting of:

- `FILE_START`
- `FILE_CHUNK`
- `FILE_END`

Each transfer is identified by a unique identifier to prevent interference between simultaneous transfers.

- `send file [client name] {file path}`

```
send file anele "C:\Users\Anele Z Ngxongo\Pictures\EE2042 Q12a.png"
File sent to anele: EE2042 Q12a.png
```

UDP Notification Registration

Clients register their UDP endpoint using:

BEEP_UDP_PORT: [port]

```
[Beep] anele sent a message in GROUP:sportcentre
```

This allows the server to send notification packets when messages are received.

Utility Commands

Additional commands allow users to retrieve system information:

- [online clients](#)

```
online clients
Online clients: hope, anele, lwandile
```

To retrieve users that are currently on active connection with the central server.

- [my groups](#)

```
my groups
Groups: sportcentre
```

To retrieve the groups the user is a member of, created and invited groups.

- [my private chats](#)

```
my private chats
Private chats: hope
```

To retrieve the list of private chats that currently exist.

- [Exit](#)

```
exit
Connection closed
```

To exit the Chat77 system and keeping the user authentication information stored.

Key Function Implementations

Authentication

The server implements authentication using a structured state machine. The authentication process interacts with the database module to verify user credentials or register new users. The server also prevents duplicate logins by checking whether an alias is already associated with an active connection.

Private Messaging Handshake

Private messaging is implemented using an application-layer handshake protocol.

First, a client sends a connection request to another user. The server records this request and notifies the recipient if they are online. The recipient may then accept the request, which establishes a p2p private connection.

This mechanism ensures that both participants explicitly agree before private communication is permitted.

Group Chat Management

Group communication is managed through a server-maintained dictionary that maps group names to sets of member aliases.

When a group is created, the creator becomes the group owner and is responsible for inviting new members. Invitations are stored in a pending invitation list until accepted by the target user.

Group messages are distributed by iterating through the list of members and sending the message to each active client.

File Transfer

File transfer in the Chat77 system is implemented using a chunk-based application-layer protocol. When a user sends a file, the sender reads the file in binary mode and divides it into fixed-size segments of 400 bytes. Each segment is encoded using Base64 encoding so that binary data can be safely transmitted within the system's text-based messaging protocol. The encoded chunks are then transmitted sequentially to the receiver through the server. The receiving client stores each chunk until the entire file has been received, after which the chunks are decoded and reassembled to reconstruct the original file on disk.

If some chunks are not received successfully, the receiver can request retransmission of the missing segments to ensure that the file is reconstructed correctly.

Wireshark and Terminal Captures

```
Enter server IP address: 127.0.0.1
Choose auth mode (REGISTER/LOGIN): login
Enter alias name: anele
Enter password: anele
Authentication successful, Welcome to Chat77!
User Commands:
To broadcast text - bdcst txt (your message)
To check for online users - online clients
To connect with a user - connect to [client]
To accept connection - accept connection [client]
To reject connection - reject connection [client]
To list private chats - my private chats
To send private text - private txt [client] (your message)
To create group - create group [group_name]
To invite to group - invite group [group_name] [client]
To accept group invite - accept group [group_name]
To reject group invite - reject group [group_name]
To list your groups - my groups
To send group text - group txt [group_name] (your message)
To send file over TCP - send file [client] [file_path]
To end one private chat - end private [client]
To exit chat77 :- exit
anele has connected to the chatroom
you are now connected
```

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	56	53075 → 22081 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
2	0.000010000	127.0.0.1	127.0.0.1	TCP	56	22081 → 53075 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
3	0.000102100	127.0.0.1	127.0.0.1	TCP	44	53075 → 22081 [ACK] Seq=1 Ack=1 Win=65280 Len=0
4	0.000047400	127.0.0.1	127.0.0.1	TCP	77	22081 → 53075 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=33
5	0.000000200	127.0.0.1	127.0.0.1	TCP	44	53075 → 22081 [ACK] Seq=1 Ack=34 Win=65280 Len=0
6	12.472275600	127.0.0.1	127.0.0.1	TCP	56	56090 → 22081 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
7	12.472336200	127.0.0.1	127.0.0.1	TCP	56	22081 → 56090 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
8	12.472413900	127.0.0.1	127.0.0.1	TCP	44	56090 → 22081 [ACK] Seq=1 Ack=1 Win=65280 Len=0
9	35.892057100	127.0.0.1	127.0.0.1	TCP	58	53075 → 22081 [PSH, ACK] Seq=1 Ack=34 Win=65280 Len=6
10	35.892063300	127.0.0.1	127.0.0.1	TCP	44	22081 → 53075 [ACK] Seq=34 Ack=7 Win=65280 Len=0
12	35.892177800	127.0.0.1	127.0.0.1	TCP	52	22081 → 53075 [PSH, ACK] Seq=34 Ack=7 Win=65280 Len=7
13	35.892265200	127.0.0.1	127.0.0.1	TCP	44	53075 → 22081 [ACK] Seq=7 Ack=41 Win=65280 Len=0
13	36.952241800	127.0.0.1	127.0.0.1	TCP	49	53075 → 22081 [PSH, ACK] Seq=7 Ack=41 Win=65280 Len=5
14	36.952298000	127.0.0.1	127.0.0.1	TCP	44	22081 → 53075 [ACK] Seq=41 Ack=12 Win=65280 Len=0
15	36.952506800	127.0.0.1	127.0.0.1	TCP	54	22081 → 53075 [PSH, ACK] Seq=41 Ack=12 Win=65280 Len=10

> Frame 11: Packet, 51 bytes on wire (408 bits), 51 bytes captured (408 bits) on Interface Dev1
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dest: 127.0.0.1
> Transmission Control Protocol, Src Port: 22081, Dest Port: 53075, Seq: 34, Ack: 7, Len: 7
Data (7 bytes)
[Data: 424c045133f8a
[Length: 7]

0000 02 00 00 00 45 00 00 2f c0 cc 40 00 00 32 faE...@...2
0020 7f 00 00 01 7f 00 00 01 56 41 cf 53 00 c4 34 c2VA...S...4
0030 6c bf ea 93 58 10 00 ff 7d 88 00 00 11 6c 63 41 1...P...}...N37
0040 53 3f 04 37

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	56	53075 → 22081 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 IS=256 SACK_PERM=0
2	0.000000000	127.0.0.1	127.0.0.1	TCP	56	22081 → 53075 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 IS=256 SACK_PERM=0
3	0.000000000	127.0.0.1	127.0.0.1	TCP	44	53075 → 22081 [ACK] Seq=1 Ack=1 Win=65280 Len=0
4	0.000000000	127.0.0.1	127.0.0.1	TCP	77	22081 → 53075 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=33
5	0.000000000	127.0.0.1	127.0.0.1	TCP	44	53075 → 22081 [ACK] Seq=1 Ack=34 Win=65280 Len=0
6	12.022225600	127.0.0.1	127.0.0.1	TCP	56	56090 → 22081 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 IS=256 SACK_PERM=0
7	12.022225600	127.0.0.1	127.0.0.1	TCP	56	22081 → 56090 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 IS=256 SACK_PERM=0
8	12.022225600	127.0.0.1	127.0.0.1	TCP	44	56090 → 22081 [ACK] Seq=1 Ack=1 Win=65280 Len=0
9	15.000000000	127.0.0.1	127.0.0.1	TCP	50	53075 → 22081 [PSH, ACK] Seq=1 Ack=34 Win=65280 Len=6
10	15.000000000	127.0.0.1	127.0.0.1	TCP	44	22081 → 53075 [ACK] Seq=34 Ack=7 Win=65280 Len=0
11	15.000000000	127.0.0.1	127.0.0.1	TCP	51	22081 → 53075 [PSH, ACK] Seq=34 Ack=7 Win=65280 Len=7
12	15.000000000	127.0.0.1	127.0.0.1	TCP	44	53075 → 22081 [ACK] Seq=7 Ack=1 Win=65280 Len=0
13	16.952225600	127.0.0.1	127.0.0.1	TCP	49	53075 → 22081 [PSH, ACK] Seq=7 Ack=41 Win=65280 Len=5
14	16.952225600	127.0.0.1	127.0.0.1	TCP	44	22081 → 53075 [ACK] Seq=41 Ack=12 Win=65280 Len=0
15	16.952225600	127.0.0.1	127.0.0.1	TCP	54	22081 → 53075 [PSH, ACK] Seq=41 Ack=12 Win=65280 Len=10

Figure 3: Authentication - Terminal protocol and Wireshark captures

```
Online clients: anele, hope, lwandile
connect to hope
Invalid command. Use: bdct txt {your message}, online clients, connect to [client], accept connection [client], reject c
onnection [client], my private chats, private txt [client] {your message}, create group [group_name], invite group [grou
p_name] [client], accept group [group_name], reject group [group_name], my groups, group txt [group_name] [message], sen
d file [client] [file_path], end private [client], or exit
connect to hope
INFO: Connection request sent to hope
Private chat connected with hope. Use: private txt hope {your message}
[Private:anele] hope: good morning(pink heart imoji)
[Beep] hope sent a message in PRIVATE
```

111	155.391110500	127.0.0.1	127.0.0.1	TCP	92	53075 → 22081 [PSH, ACK] Seq=99 Ack=414 Win=65024 Len=48
112	155.391217400	127.0.0.1	127.0.0.1	TCP	44	22081 → 53075 [ACK] Seq=414 Ack=147 Win=65280 Len=0
113	155.391533900	127.0.0.1	127.0.0.1	TCP	97	22081 → 56090 [PSH, ACK] Seq=337 Ack=92 Win=65280 Len=53
114	155.391634800	127.0.0.1	127.0.0.1	TCP	44	56090 → 22081 [ACK] Seq=92 Ack=390 Win=65024 Len=0
115	155.392859300	127.0.0.1	127.0.0.1	UDP	50	57429 → 51151 Len=18

Figure 4: Private Text

```
create group sportcentre
You joined group: sportcentre
invite group sportcentre hope
INFO: Group invite sent to hope for sportcentre
INFO: hope joined group sportcentre
invite group sportcentre lwandile
INFO: Group invite sent to lwandile for sportcentre
INFO: lwandile joined group sportcentre
[Group:sportcentre] lwandile: where are you guys?
[Beep] lwandile sent a message in GROUP:sportcentre
```

No.	Time	Source	Destination	Protocol	Length	Info
189	720.836126300	127.0.0.1	127.0.0.1	TCP	44	56090 → 22081 [ACK] Seq=117 Ack=525 Win=64768 Len=0
190	720.836126300	127.0.0.1	127.0.0.1	TCP	81	22081 → 53150 [PSH, ACK] Seq=446 Ack=169 Win=65280 Len=37
191	720.836208400	127.0.0.1	127.0.0.1	TCP	44	53150 → 22081 [ACK] Seq=169 Ack=448 Win=65024 Len=0
192	720.836364900	127.0.0.1	127.0.0.1	TCP	69	22081 → 53075 [PSH, ACK] Seq=487 Ack=172 Win=65280 Len=25
193	720.836424300	127.0.0.1	127.0.0.1	TCP	44	53075 → 22081 [ACK] Seq=172 Ack=512 Win=65024 Len=0
194	756.629792000	196.24.188.217	196.24.191.255	BROWSER	248	Domain/workgroup Announcement WORKGROUP, NT Workstation, Domain Enum
195	760.337432100	127.0.0.1	127.0.0.1	TCP	82	53075 → 22081 [PSH, ACK] Seq=172 Ack=512 Win=65024 Len=38
196	760.337488100	127.0.0.1	127.0.0.1	TCP	44	22081 → 53075 [ACK] Seq=512 Ack=210 Win=65280 Len=0
197	760.337604000	127.0.0.1	127.0.0.1	TCP	87	22081 → 56090 [PSH, ACK] Seq=525 Ack=117 Win=65280 Len=43
198	760.337651700	127.0.0.1	127.0.0.1	TCP	44	56090 → 22081 [ACK] Seq=117 Ack=568 Win=64768 Len=0
199	760.337769200	127.0.0.1	127.0.0.1	UDP	60	57429 → 51151 Len=28
200	760.337892200	127.0.0.1	127.0.0.1	TCP	87	22081 → 53150 [PSH, ACK] Seq=483 Ack=169 Win=65280 Len=43
201	760.337929300	127.0.0.1	127.0.0.1	TCP	44	53150 → 22081 [ACK] Seq=169 Ack=526 Win=64768 Len=0
202	760.338020500	127.0.0.1	127.0.0.1	UDP	60	57429 → 64895 Len=28

Figure 5: Group Text

```
[Group:sportcentre] lwandile: where are you guys?  
[Beep] lwandile sent a message in GROUP:sportcentre
```

No.	Time	Source	Destination	Protocol	Length	Info
189	720.386126380	127.0.0.1	127.0.0.1	TCP	44	56900 → 22081 [ACK] Seq=117 Ack=525 Min=64768 Len=0
190	720.836126400	127.0.0.1	127.0.0.1	TCP	81	22081 → 53150 [PSH, ACK] Seq=446 Ack=169 Min=65280 Len=37
191	720.836241000	127.0.0.1	127.0.0.1	TCP	44	53150 → 22081 [ACK] Seq=169 Ack=483 Min=65804 Len=0
192	720.836349000	127.0.0.1	127.0.0.1	TCP	69	22081 → 53075 [PSH, ACK] Seq=487 Ack=172 Min=65280 Len=25
193	720.836423400	127.0.0.1	127.0.0.1	TCP	44	53075 → 22081 [ACK] Seq=172 Ack=512 Min=65804 Len=0
194	750.629792000	196.24.191.25	196.24.191.25	BROWSER	248	DomainWorkgroup Announcement WORKGROUP, NT Workstation, Domain Enum
195	760.337432100	127.0.0.1	127.0.0.1	TCP	82	53075 → 22081 [PSH, ACK] Seq=172 Ack=512 Min=65804 Len=38
196	760.337481800	127.0.0.1	127.0.0.1	TCP	44	22081 → 53075 [ACK] Seq=512 Ack=210 Min=65280 Len=0
197	760.337608400	127.0.0.1	127.0.0.1	TCP	87	22081 → 56900 [ACK] Seq=525 Ack=117 Min=65280 Len=43
198	760.337617700	127.0.0.1	127.0.0.1	TCP	44	56900 → 22081 [ACK] Seq=117 Ack=508 Min=64768 Len=0
199	760.337799200	127.0.0.1	127.0.0.1	UDP	60	57429 → 51551 Len=20
200	760.337802200	127.0.0.1	127.0.0.1	TCP	87	22081 → 53150 [PSH, ACK] Seq=483 Ack=169 Min=65280 Len=43
201	760.337923900	127.0.0.1	127.0.0.1	TCP	44	53150 → 22081 [ACK] Seq=169 Ack=526 Min=64768 Len=0
202	760.338029500	127.0.0.1	127.0.0.1	UDP	60	57429 → 64895 Len=20

Frame 199: Packet, 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface lve
Ethernet II, Src: Intel(R) Gigabit Ethernet Controller (08:00:27:00:00:00), Dst: Intel(R) Gigabit Ethernet Controller (08:00:27:00:00:00)

Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

User Datagram Protocol, Src Port: 57429, Dst Port: 51551

Data (28 bytes)
Data: 42454553a61e65c6c53e4752af55503a73706f72726f3656e472d05
[Length: 28]

Figure 6: UDP BEEP notification

```
[Beep] anele sent a message in FILE
File received from anele: C:\Users\Anele Z Nxgongo\OneDrive - University of Cape Town\UCT_COURSES\2026\CSC3002F\Assignme
nt 1\Networks\downloads\EE2042_Q12a.png
```

[illegible]

Figure 7: File Transfer

Implementation Challenges

We had a lot of implementation challenges in this assignment, as this was a learning curve. These are the main challenges we faced:

- Managing concurrent client safely
At first, we were using one thread per client, which makes race condition possible when more than one client is active. We implemented multiple threads to run specific features to prevent race-condition
- Keeping protocol parsing reliable
It was a challenge to come up with prefix protocols that are user-friendly and creative. But we thrived through that challenge that now we have good and easy to-use prefix protocols as seen above in the 'key functionality implementation' subtopic
- Implementing file transfer over the chat protocol
We could not figure out the destination for the file chunks after the transfer, but then we ended up implementing a solution; we created a download directory for the new file.
- Maintaining consistent state cleanup
When a client exits unexpectedly, we cannot clean up after the client's track record, but that was fixed with functions that handle those inconveniences

- **Synchronization Between GUI and Network Threads**
One of the main challenges encountered was managing communication between the GUI and the server while keeping the interface responsive. The chat application relies on continuous network communication using sockets, but GUI frameworks such as Tkinter operate on a single main event loop. If network operations run on the main thread, the interface freezes.
- **File Transfer Integration on the GUI**
Adding file transfer functionality on the GUI was tedious and required significant adjustments to both the interface and the communication protocol. The GUI needed a dedicated button for file transfer and a way to encode and send files through the server. Since files were transmitted as encoded chunks, the GUI also needed logic to reconstruct and save the file correctly on the receiving side.

Performance and Reliability Discussion

The Chat77 application performs reliably under typical operating conditions involving several concurrent users. The system uses Transmission Control Protocol as the primary communication mechanism, ensuring that chat messages and file transfers are delivered reliably and in the correct order. TCP provides mechanisms such as sequence numbers, acknowledgments, and retransmission timers to guarantee reliable data transfer, ensuring that data received by the application is complete, correctly ordered, and free from duplication.

System latency is generally low, particularly within local network environments. Messages received by the server are forwarded immediately to their intended client, enabling near real-time communication between users. Responsiveness is further enhanced using User Datagram Protocol notifications. UDP provides a lightweight transport service that does not require connection establishment and introduces minimal overhead, making it suitable for simple notification alerts such as message beeps.

Despite these strengths, the system has certain limitations. Chat messages are primarily stored in memory during active sessions, meaning that all messages are lost if the server restarts unexpectedly. Future improvements could include implementing persistent message storage using a database, adopting more scalable server to improve performance and fault tolerance.

Basic Security Considerations

Although the Chat77 system includes authentication mechanisms, several security risks remain in the current implementation.

One major vulnerability is the lack of encryption. All communication between clients and the server is transmitted in plaintext allowing an attacker monitoring the network to intercept sensitive information such as login credentials and private messages.

This issue could be addressed by implementing Transport Layer Security, which would encrypt all TCP communication and UDP BEEP.

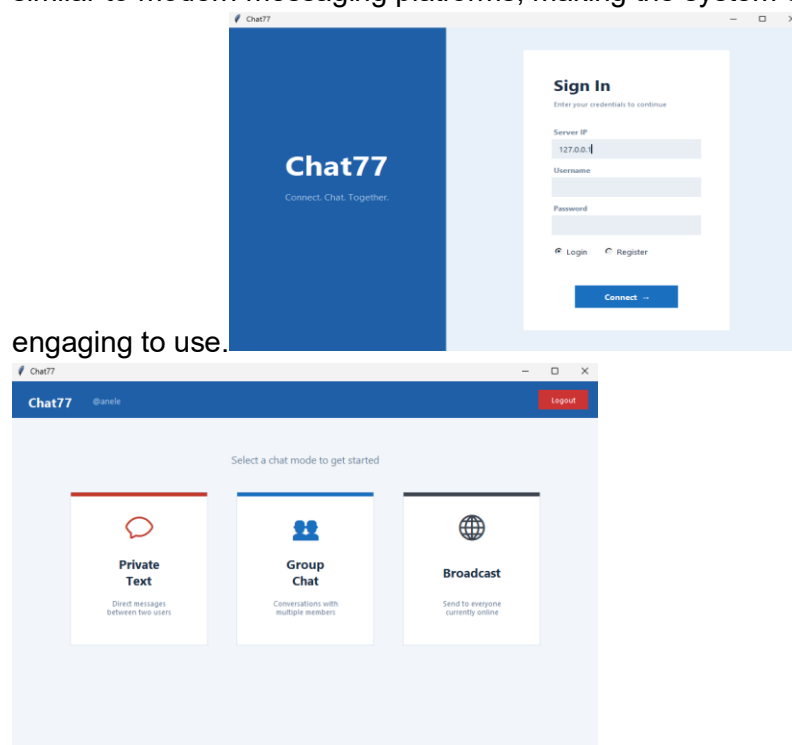
Another concern relates to password storage. If passwords are stored in plaintext within the database, a database compromise could expose user credentials. A more secure approach would involve hashing passwords using cryptographic hash functions ensuring that passwords are not easily traceable.

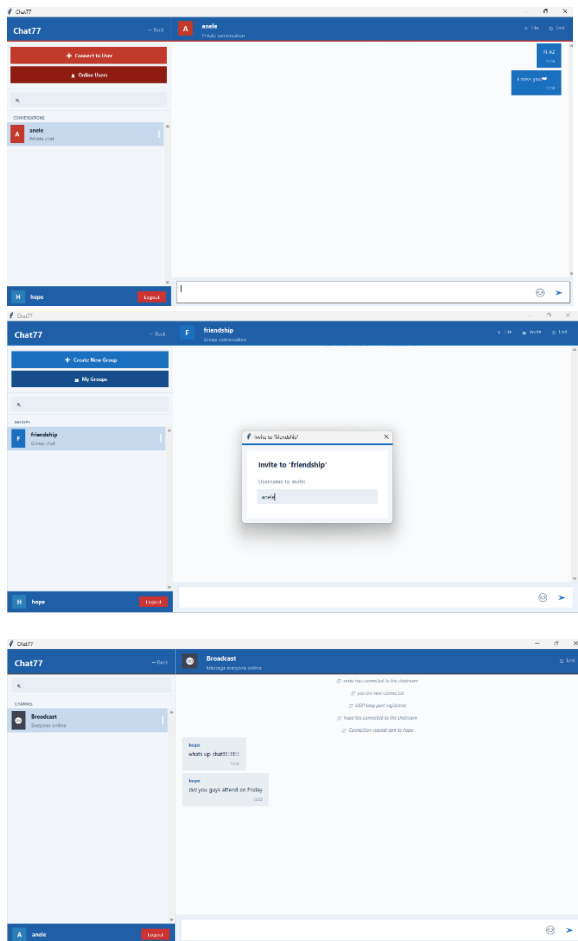
Future versions of the system could improve security by implementing encrypted communication, secure password hashing, rate limiting to control excessive requests, and stronger authentication mechanisms to better protect both user data and system resources.

Creativity

A graphical user interface was implemented with the goal of improving usability and accessibility for users. While a command-line interface could provide the same core functionality, a GUI offers a more intuitive and user-friendly environment for interacting with the chat system. This design choice allows users to interact with the application in a manner similar to modern messaging platforms, making the system easier to learn and more

engaging to use.





From a design perspective, the GUI also helps organize different features of the application in a clear and structured way. Elements such as separate areas for private text, groups and broadcast. Also, in each elements separations such as displaying messages, entering text, send file icon, emojis icon and showing notifications allow users to quickly understand the state of the conversation and respond efficiently. Additionally, graphical notifications and sound alerts improve responsiveness by immediately informing users when new messages arrive.

Conclusion

In conclusion, the Chat77 project successfully demonstrates the design and implementation of a networked chat application built on fundamental networking principles. The system integrates Transmission Control Protocol for reliable communication and User Datagram Protocol for lightweight notifications, enabling efficient and responsive interaction between users. Through the development of a custom application-layer protocol, the application supports essential communication features such as authentication, private messaging, group chat, and media file transfer. The hybrid client - server architecture allows a central server to coordinate communication and manage user sessions while client applications provide an interactive graphical interface. While the system performs effectively under typical conditions, certain limitations remain, including scalability constraints associated with the thread-per-client architecture and limited support for message persistence and security. Future improvements could include the introduction of encrypted communication using TLS, secure password hashing, persistent message storage, and more scalable networking

architectures. Overall, the Chat77 application provides a practical demonstration of socket programming, concurrent server design and application-layer protocol development, forming a solid foundation for future enhancements and further exploration of networked systems.